Code Layout, Readability and Reusability

Date	21 October 2026
Team ID	TDS-AI-2026
Project Name	Personalized Travel Itinerary Planner
Maximum Marks	5 Marks

Project Title

Personalized Travel Itinerary Planner

Project Directory Structure

```
Personalized Travel Itinerary Planner/
■■■ backend/
■   ■■■ app/
■   ■   ■■■ __init__.py
■   ■   ■■■ main.py                    # FastAPI app entry point
■   ■   ■■■ core/
■   ■   ■   ■■■ config.py              # Settings (Pydantic BaseSettings)
■   ■   ■   ■■■ security.py            # JWT verification via JWKS
■   ■   ■   ■■■ supabase.py            # Per-user Supabase client factory
■   ■   ■   ■■■ errors.py              # Centralized exception handlers
■   ■   ■■■ routers/
■   ■   ■   ■■■ health.py              # GET /api/health
■   ■   ■   ■■■ me.py                  # GET /api/me
■   ■   ■   ■■■ trips.py               # Trip CRUD + AI endpoints
■   ■   ■   ■■■ activities.py          # Activity PATCH + DELETE
■   ■   ■■■ schemas/
■   ■   ■   ■■■ trip.py                # Trip Pydantic models
■   ■   ■   ■■■ day.py                 # Day Pydantic models
■   ■   ■   ■■■ activity.py            # Activity Pydantic models
■   ■   ■■■ services/
■   ■   ■   ■■■ trip_service.py        # Business logic layer
■   ■   ■   ■■■ generation_service.py  # AI generation orchestration
■   ■   ■   ■■■ ai_service.py          # LLM API calls + prompt engineering
■   ■   ■■■ repositories/
■   ■   ■   ■■■ trip_repo.py           # Database access layer
■   ■   ■■■ prompts/                   # Reserved for prompt templates
■   ■■■ migrations/
■   ■   ■■■ 001_init.sql               # Full database schema + RLS
■   ■■■ tests/
■   ■   ■■■ test_schemas.py            # Schema validation tests
■   ■   ■■■ test_trip_service.py       # Service layer tests
■   ■■■ requirements.txt
■   ■■■ .env.example
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ App.tsx                    # Root component with routing
■   ■   ■■■ main.tsx                   # React entry point
■   ■   ■■■ App.css / index.css        # Global styles
■   ■   ■■■ api/
■   ■   ■   ■■■ client.ts              # HTTP client with auth headers
■   ■   ■■■ auth/
■   ■   ■   ■■■ AuthProvider.tsx       # Supabase session context
■   ■   ■   ■■■ ProtectedRoute.tsx     # Auth guard component
■   ■   ■   ■■■ useAuth.ts             # Auth hook
■   ■   ■■■ components/
■   ■   ■   ■■■ ActivityEditDialog.tsx # Activity edit modal
■   ■   ■   ■■■ ui/                    # shadcn/ui components
■   ■   ■■■ layouts/
```

```
■   ■   ■   ■■■ AppShell.tsx          # App layout with nav
■   ■   ■■■ lib/
■   ■   ■   ■■■ supabase.ts          # Supabase client init
■   ■   ■■■ pages/
■   ■   ■   ■■■ LoginPage.tsx         # Login form
■   ■   ■   ■■■ SignupPage.tsx        # Registration form
■   ■   ■   ■■■ DashboardPage.tsx     # Trip list
■   ■   ■   ■■■ NewTripPage.tsx       # Trip creation form
■   ■   ■   ■■■ TripPage.tsx          # Itinerary detail view
■   ■   ■■■ types/
■   ■       ■■■ trip.ts              # TypeScript interfaces
■   ■■■ package.json
■   ■■■ tsconfig.json
■   ■■■ vite.config.ts
■   ■■■ vercel.json
■■■ scripts/
■   ■■■ generate_docs.py             # Documentation generator
■   ■■■ convert_to_pdf.py            # Markdown to PDF converter
■■■ docs/
    ■■■ decisions.md                 # Architecture decision records
```

Code Design Principles

1. Separation of Concerns

- **Routers**: Handle HTTP request/response only — no business logic
- **Services**: Contain all business logic (trip_service, generation_service, ai_service)
- **Repositories**: Handle raw database queries — single source of truth for data access
- **Schemas**: Define data shapes for validation, serialization, and documentation

2. Dependency Injection (FastAPI)

```
@router.post("", response_model=TripOut, status_code=201)
def create_trip(
    payload: TripCreate,                          # Auto-validated request body
    user: AuthUser = Depends(get_current_user),     # JWT verification
    db: Client = Depends(get_user_supabase),        # RLS-scoped DB client
):
    return trip_service.create_trip(db, user.id, payload)
```

3. Type Safety

- **Backend**: Pydantic models with field validators for all inputs/outputs
- **Frontend**: TypeScript interfaces + Zod schemas for runtime validation
- Both ensure compile-time and runtime type safety

4. Reusable Validators

```
# Shared across TripCreate and TripUpdate
def validate_date_range(start: date, end: date) -> None:
def normalise_currency(value: str) -> str:
def clean_interests(values: list[str]) -> list[str]:
```

5. Error Handling

- Centralized exception handlers in errors.py
- Consistent HTTP error responses with meaningful messages
- Frontend ApiError class for typed error handling

6. Pure Functions for Testing

```
# Tested independently without database
def compute_day_cost(activities: list[dict]) -> float:
def compute_trip_total(days: list[dict]) -> float:
def build_trip_detail(trip: dict, days_raw: list[dict]) -> TripDetailOut:
```